

NUMA Binding for Fusion Compiler-Class Physical Design Jobs

White paper for the numa-fc-bind project

Scope: Align CPU affinity and DRAM allocation to one NUMA node for a single fat PD job when the working set fits that node's free memory.

Companion: working code, hardware-only compare harness, mechanism GIF under numa-fc-bind/.

Abstract

Physical-design tools such as Synopsys Fusion Compiler allocate multi-gigabyte working sets and spawn many threads. On dual- and quad-socket Linux hosts, memory is NUMA-partitioned: each socket owns cores and DRAM. Cross-socket accesses traverse an interconnect (Intel UPI, AMD Infinity Fabric) and pay latency and bandwidth tax. Because Linux often places pages by **first-touch**, and schedulers may migrate threads, a job can run with busy CPUs on one node and hot pages on another while `top` still looks healthy.

The operational fix—when peak RSS fits one node's free RAM—is:

```
numactl --cpunodebind=N --membind=N <tool> ...
```

This paper documents the failure mode, the policy knobs, the safety model, the implementation in `numa-fc-bind`, and what may honestly be claimed from measurement. Single-node hosts cannot demonstrate remote DRAM; the project refuses to fabricate that contrast.

Table of contents

1. [Motivation](#)
2. [Problem statement](#)
3. [NUMA and first-touch](#)
4. [Policy](#)
5. [Safety](#)
6. [Implementation](#)
7. [Measurement](#)
8. [Mechanism GIF](#)
9. [When it helps](#)
10. [Claims policy](#)
11. [Conclusion](#)
12. [Appendix: commands](#)

1. Motivation

PD farm nodes are often 2S/4S machines. Teams scale thread counts and wonder why turnaround stalls. A recurring root cause is **memory locality**, not algorithmic throughput: remote fills on the interconnect. The DeepSeek-style ops note—“pin FC to one NUMA node and bind memory there”—is directionally correct. It needs an operable wrapper, a honest measurement path, and clear limits.

A sibling concern is filesystem locality (NFS vs tmpfs). That is a different lever. This paper addresses DRAM placement only.

2. Problem statement

Given a PD tool command on a multi-socket Linux host, place its threads and anonymous memory on the same NUMA node when safe, without changing the tool binary or functional results, and without hard-binding into out-of-memory.

Success criteria:

1. Functional equivalence (same scripts, same deliverables).
 2. On ≥ 2 -node hardware: measurable microbench contrast (local vs remote membind) and/or improved stage wall time with healthier numastat.
 3. Wrapper refuses unsafe hard membind when MemFree is too low.
 4. No citeable “speedup” artifacts from single-node hosts.
-

3. NUMA and first-touch

NUMA = Non-Uniform Memory Access. Local DRAM is cheaper than remote DRAM.

First-touch: a newly written anonymous page is typically backed on the node of the CPU that first wrote it. Patterns that create remote access:

1. Master thread initializes a huge heap on node 0; workers later run on node 1.
2. Threads bounce across sockets while the heap stays put.
3. Unbound processes share a machine and scatter pages under load.

AutoNUMA can migrate pages, but not reliably for short phases or huge stable working sets. Production overnight runs benefit from explicit policy.

Discovery:

```
./scripts/numa_report.sh
numactl -H
```

4. Policy

Flag	Meaning
--cpunodebind=N	Restrict to node N CPUs
--membind=N	Allocate only from node N (hard)
--preferred=N	Prefer node N; spill if needed
--localalloc	Prefer the touching CPU's node
--interleave=...	Stripe pages (sometimes for machine-sized heaps)

Primary recipe (job fits node 0):

```
./scripts/run_with_numactl.sh 0 fc_shell -f run.tcl
```

Use POLICY=preferred when peak RSS is uncertain.

5. Safety

Hard membind on a node with insufficient free memory causes allocation failure or reclaim storms. The wrapper:

1. Prints per-node MemTotal / MemFree and full numactl -H.
2. Refuses membind if MemFree < MIN_MEMFREE_GIB (default 8) unless FORCE=1.
3. Supports POLICY=preferred as a softer alternative.
4. Warns when the host has only one NUMA node.

Rule of thumb: peak RSS × safety factor must fit the target node's free RAM.

6. Implementation

Path	Role
scripts/numa_report.sh	Topology + recommendation
scripts/run_with_numactl.sh	Production wrapper
src/numa_mem_bench.cpp	STREAM triad + pointer chase
examples/compare_numa.sh	Hardware remote vs local (median of trials)
tools/numactl-root/	Vendored numactl / numastat
demo/gen_gif.py	Mechanism GIF generator

compare_numa.sh exits with status 2 on single-node hosts and writes CLAIM_GATE=no. It does not synthesize remote latency.

7. Measurement

7.1 Microbench

Metric	Definition	Better
triad	STREAM triad GiB/s	higher
chase	pointer-chase ns/hop	lower

Aggregate: median of TRIALS (default 5), with min/max spreads printed so noise is visible.

7.2 Citeability

Situation	Publicly cite ratios?
Hardware compare, CLAIM_GATE=CITEABLE=yes	Yes — as microbench under numactl , not FC wall time
1-node host	No — script refuses
FC stage wall time + numastat -p	Yes — strongest farm evidence

7.3 This document's CI / cloud context

Many agent VMs have one NUMA node. There is **no hardware remote-vs-local result** to publish from such a host. Absence of a speedup table is correct.

8. Mechanism GIF

demo/numa_fc_bind.gif illustrates the failure mode and the policy. It is labeled **mechanism** • **not a benchmark**. It does not display fabricated percentage speedups. Regenerate with `make gif`.

9. When it helps

Situation	Expectation
2S/4S, one FC, RSS fits node free RAM	Primary target
1 NUMA node	No DRAM-locality win from membind
RSS exceeds node free RAM + hard membind	Harm
Bottleneck is NFS / disk	Fix I/O tiering first
Many small jobs	Prefer one heavy job per node

10. Claims policy

1. Do not post X× numbers without topology proof (≥ 2 nodes) and CLAIM_GATE.
 2. Do not treat GIF or microbench as FC wall-clock.
 3. Do not hard-bind into OOM and call it optimization.
 4. Prefer measured farm evidence: stage time + numastat.
-

11. Conclusion

NUMA binding is memory **placement**, not a new PD algorithm. It is useful on multi-socket farms when the job fits one node, dangerous when it does not, and unmeasurable as remote-vs-local on single-node hosts. numa-fc-bind is built to be operable on a farm and honest in constrained environments.

Appendix: commands

```
cd numa-fc-bind
./scripts/numa_report.sh
make -j && make smoke
make gif && make pdf
./scripts/run_with_numactl.sh 0 fc_shell -f run.tcl
TRIALS=5 BYTES=1G ./examples/compare_numa.sh    #  $\geq 2$  nodes
```